



**COMPOSABLE
SECURITY**

REPORT

Smart contract security review for Lido

Prepared by: Composable Security

Report ID: LDO-55729910

Test time period: 2026-08-17 - 2026-08-20

Retest time period: 2026-08-25 - 2026-08-26

Report date: 2026-08-26

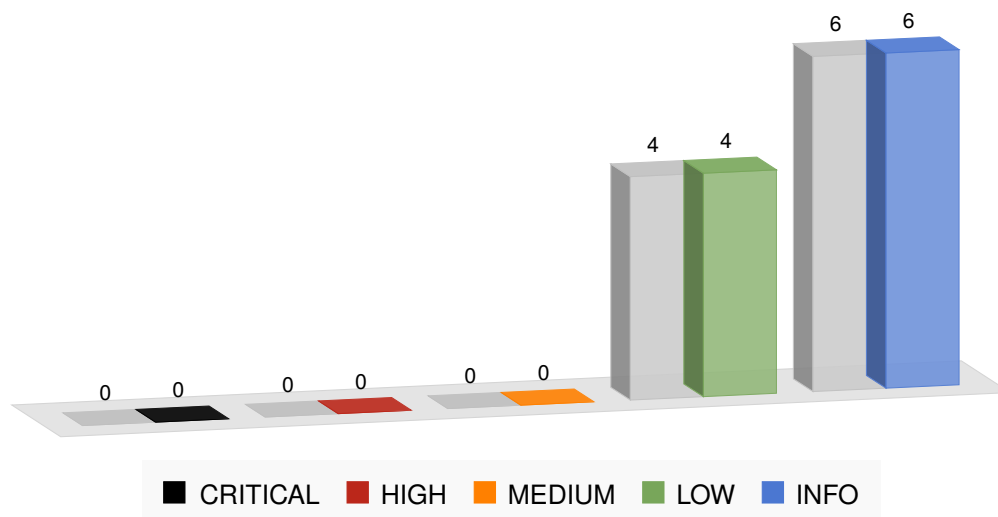
Version: 1.0

Visit: composable-security.com

Contents

1. Retest summary (2026-08-26)	2
1.1 Results	2
1.2 Scope	2
2. Current findings status	4
3. Security review summary (2026-08-20)	5
3.1 Client project	5
3.2 Results	6
3.3 Scope	6
4. Project details	7
4.1 Projects goal	7
4.2 Agreed scope of tests	7
4.3 Threat analysis	7
4.4 Testing methodology	9
4.5 Disclaimer	9
5. Vulnerabilities	10
[LDO-55729910-L01] Pending nominations cannot be cancelled independently . .	10
[LDO-55729910-L02] Codeless targets report successful execution	11
[LDO-55729910-L03] One-step ownership transfer can lock administration	12
[LDO-55729910-L04] Pause signatures can be replayed after unpausing	14
6. Recommendations	16
[LDO-55729910-R01] Bound the delegate cooldown	16
[LDO-55729910-R02] Require the delegation contract interface for guardians	17
[LDO-55729910-R03] Constrain delegate signature semantics	18
[LDO-55729910-R04] Update the ERC-1271 dependency	19
[LDO-55729910-R05] Emit an event for delegated executions	20
[LDO-55729910-R06] Validate and accept delegate nominations	20
7. Impact on risk classification	22
8. Long-term best practices	23
8.1 Use automated tools to scan your code regularly	23
8.2 Perform threat modeling	23
8.3 Use Smart Contract Security Verification Standard	23
8.4 Discuss audit reports and learn from them	23
8.5 Monitor your and similar contracts	23

1. Retest summary (2026-08-26)



The description of the current status for each retested vulnerability and recommendation has been added in its section.

1.1. Results

The **Composable Security** team participated in a one-time iteration to verify if the vulnerabilities detected during the tests (between 2026-08-17 and 2026-08-20) were correctly removed and no longer appear in the code.

The current status of detected issues is as follows:

- **All issues were acknowledged and not fixed within this release.** The team considers the identified areas for improvement to be cosmetic in nature and not to introduce security risk. Based on this assessment, they decided to proceed with the current implementation rather than redeploying it for all delegators. The reported issues will be taken into account in future updates.
- During the retest, the risk rating of 1 vulnerability (LDO-55729910-L01) was reduced from originally reported **medium** to **low** after further discussion with the Lido team and reassessment of its practical impact.

1.2. Scope

The retest scope included the same contracts, on the same commit in the same repository.

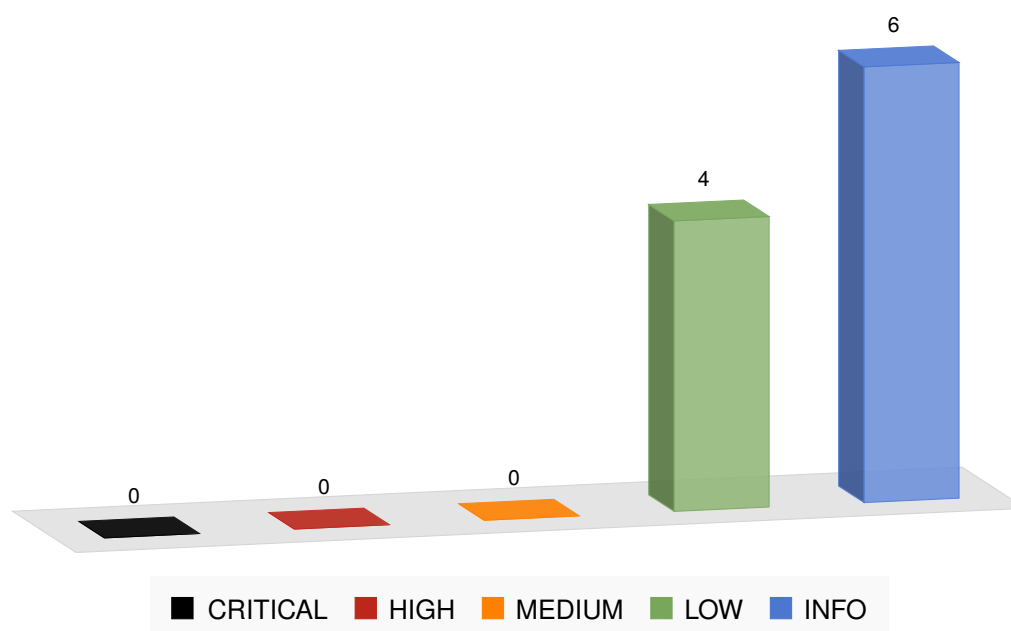
GitHub repository: <https://github.com/lidofinance/execution-delegation-framework/>

CommitID: 557299104ad3eb1a74198933bd016328c490e276

2. Current findings status

ID	Severity	Vulnerability	Status
LDO-55729910-L01	LOW	Pending nominations cannot be cancelled independently	ACKNOWLEDGED
LDO-55729910-L02	LOW	Codeless targets report successful execution	ACKNOWLEDGED
LDO-55729910-L03	LOW	One-step ownership transfer can lock administration	ACKNOWLEDGED
LDO-55729910-L04	LOW	Pause signatures can be replayed after unpausing	ACKNOWLEDGED
ID	Severity	Recommendation	Status
LDO-55729910-R01	INFO	Bound the delegate cooldown	ACKNOWLEDGED
LDO-55729910-R02	INFO	Require the delegation contract interface for guardians	ACKNOWLEDGED
LDO-55729910-R03	INFO	Constrain delegate signature semantics	ACKNOWLEDGED
LDO-55729910-R04	INFO	Update the ERC-1271 dependency	ACKNOWLEDGED
LDO-55729910-R05	INFO	Emit an event for delegated executions	ACKNOWLEDGED
LDO-55729910-R06	INFO	Validate and accept delegate nominations	ACKNOWLEDGED

3. Security review summary (2026-08-20)



3.1. Client project

Lido on Ethereum is a liquid-staking protocol that allows users to deposit ETH and receive stETH while the pooled ether is staked through node operators selected by the Lido DAO. The protocol's core contracts coordinate staking, withdrawals, oracle reporting, and deposit security without giving node operators direct access to user assets.

The **Execution Delegation Framework and DSM v5 integration** comprise a minimal on-chain delegation contract suite and the corresponding Lido Core changes required to place permissioned Oracle and Deposit Security Module operators behind delegation contracts.

The framework separates a protocol-authorized principal from the hot key used by an off-chain operator. Each factory-deployed `DelegationContract` has an immutable owner and one effective delegate. The owner can rotate or immediately revoke the delegate and can irreversibly terminate the instance, while the delegate can act through direct execution or signatures verified through ERC-1271. This allows operational keys to change without repeating the protocol governance process that assigned the underlying permission.

In this engagement, the Composable Security team focused on the **Execution Delegation Framework** and the **Lido Core DSM/EDF integration**. The review covered the delegation primitive and the changes in `DepositSecurityModule` version 5.

3.2. Results

The **Lido** engaged Composable Security to review security of **Execution Delegation Framework**. Composable Security conducted this assessment over 1 person-week with 2 engineers.

The summary of findings is as follows:

- No vulnerabilities with **critical** and **high** impact on risk were identified.
- **Four** vulnerabilities with a **low** impact on risk were identified.
- Six **recommendations** have been proposed that can improve overall security and help implement best practice.
- The **Lido** team plans to review each deployed delegation contract manually before adding it to guardians list and giving it any power.
- The **DelegationContract** includes a contract termination function that immediately blocks access to the contract, preventing any previously granted approvals from being revoked. However, these delegation contracts will not be used in association with any tokens or other assets supporting approvals, so this does not pose a security risk.
- The team was engaged and the communication was very good.

Composable Security recommends that **Lido** complete the following:

- Address all reported issues.
- Extend unit tests with scenarios that cover detected vulnerabilities where possible.
- Consider whether the detected vulnerabilities may exist in other places (or ongoing projects) that have not been detected during engagement.
- Review dependencies and upgrade to the latest versions.

3.3. Scope

The scope of the tests included selected contracts from the following repositories.

GitHub repository: <https://github.com/lidofinance/execution-delegation-framework/>

CommitID: 557299104ad3eb1a74198933bd016328c490e276

GitHub repository: <https://github.com/lidofinance/core>

CommitID: 8c4cee2be76a4ed8067026e30b5bdc64a4e3cb29

The detailed scope of tests can be found in Agreed scope of tests.

4. Project details

4.1. Projects goal

The Composable Security team focused during this security review on the following:

- Perform a tailored threat analysis.
- Ensure that smart contract code is written according to security best practices.
- Identify security issues and potential threats both for **Lido** and their users.
- The secondary goal is to improve code clarity and optimize code where possible.

4.2. Agreed scope of tests

The subjects of the test were selected contracts from two **Lido** repositories.

GitHub repository:

<https://github.com/lidofinance/execution-delegation-framework/>

CommitID: 557299104ad3eb1a74198933bd016328c490e276

Files in scope:

```
execution-delegation-framework/src
├── DelegationContract.sol
└── DelegationFactory.sol
```

GitHub repository:

<https://github.com/lidofinance/core>

CommitID: 8c4cee2be76a4ed8067026e30b5bdc64a4e3cb29

Files in scope:

```
core/contracts/0.8.9/
└── DepositSecurityModule.sol
```

Documentation: LIP-37: Execution Delegation Framework

4.3. Threat analysis

This section summarizes the potential threats that were identified during the initial threat modeling performed before the security review. The tests were focused, but not limited to, finding security issues that could be exploited to achieve these threats.

Key assets that require protection:

- EDF owner authority.
- Active and pending delegate state.
- Delegated push-call authority.
- Delegated ERC-1271 signature authority.
- ETH supplied to delegated calls.
- DSM guardian membership and quorum.
- Deposit availability and pause state.

Potential attackers goals:

- A malicious deployer seeks to make an unapproved delegation instance appear trusted.
- A compromised EDF owner seeks to transfer operational authority to an attacker-controlled delegate.
- A compromised EDF owner seeks to disable delegated execution and signature verification.
- A malicious delegate seeks to exercise the delegation instance's permissions for unauthorized benefit.
- A malicious delegate seeks to disrupt DSM deposits, pausing, or unvetting.
- A delegate-selected target seeks to capture ETH or prevent reliable execution.
- A compromised DSM owner seeks to take over or permanently disable DSM administration.
- Colluding guardians seek to authorize invalid buffered-deposit decisions.
- A faulty Deposit Contract seeks to make valid deposits fail or invalid state appear current.
- An anonymous attacker seeks to make invalid signatures pass or valid signatures fail.
- An anonymous attacker blocks the delegate from using the contract.

Potential scenarios to achieve the indicated attacker's goals:

- A malicious caller deploys an instance naming a victim organization as owner and an attacker key as delegate, then presents the factory event as evidence of an approved principal.
- A deployer supplies an extreme cooldown that makes future nomination arithmetic revert or schedules the replacement delegate effectively forever.
- An owner transaction intended to replace a pending delegate lands after maturity, causing `_settle` to promote the unwanted delegate before scheduling the replacement.
- An owner cancels a pending nomination through `revokeDelegate`, which also removes the working delegate and interrupts operations for a full cooldown.
- A compromised delegate calls an authorized downstream contract through `execute` and creates a persistent approval or role grant that survives later delegate revocation.
- A delegate supplies a codeless or wrong-network target, making the low-level call report success while attached ETH is transferred without executing the intended operation.
- A malicious target returns excessive data and forces `execute` to consume enough gas

to prevent a time-sensitive delegated operation from completing.

- A signature accepted for one delegation context is replayed wherever an integrator supplies the same unbound hash to `isValidSignature`.
- An external caller replays a still-valid guardian pause signature after the owner unpauses, restoring the paused state without fresh guardian intent.
- A quorum of compromised guardians signs a malicious deposit payload that passes the guardian checks and triggers a Staking Router deposit.
- A compromised guardian submits a signed unvetting payload that reduces vetted signing-key counts for targeted node operators.
- A faulty or malicious Deposit Contract (or RPC provider) supplies an unexpected deposit root that blocks current attestations or makes an attacker-selected root appear current.
- A faulty or malicious Staking Router (or RPC provider) returns manipulated nonce or deposit-distance data and causes valid deposits or unvetting actions to fail.
- A faulty signature verification allows to execute prohibited or unauthorized operations.
- Privilege escalation through incorrect access control to functions or badly written modifiers.

4.4. Testing methodology

Smart contract security review was performed using the following methods:

- Q&A sessions with the **Lido** development team to thoroughly understand intentions and assumptions of the project.
- Initial threat modeling to identify key areas and focus on covering the most relevant scenarios based on real threats.
- Automatic tests using slither.
- Custom scripts (e.g. unit tests) to verify scenarios from initial threat modeling.
- **Manual review of the code.**

4.5. Disclaimer

Smart contract security review **IS NOT A SECURITY WARRANTY**.

During the tests, the Composable Security team makes every effort to detect any occurring problems and help to address them. However, it is not allowed to treat the report as a security certificate and assume that the project does not contain any vulnerabilities. Securing smart contract platforms is a multi-stage process, starting from threat modeling, through development based on best practices, security reviews and formal verification, ending with constant monitoring and incident response.

Therefore, we encourage the implementation of security mechanisms at all stages of development and maintenance.

5. Vulnerabilities

[LDO-55729910-L01] Pending nominations cannot be cancelled independently

LOW

ACKNOWLEDGED

Retest (2026-08-25)

The vulnerability has been acknowledged as the team considers the incentive to revert the old delegate is unlikely to happen.

Lido team's response: *This scenario is mitigated by calling `nominateDelegate` again with the correct address. There is still an edge case where the owner simply changes their mind about a rotation mid-flight, but the incentive to revert to the old delegate is weak. It's still preferable to proceed to the new delegator address, since EDF is designed for that flow.*

Occurrences

- DelegationContract.sol (#L87-L96, #L100-L106)

Description

The `revokeDelegate` function clears both `_pendingDelegate` and `_currentDelegate`. The owner therefore cannot cancel an incorrect or unwanted pending nomination while retaining the working delegate. Restoring service requires another nomination followed by the full cooldown.

The cancellation is also not represented accurately in the event stream. The `DelegateRevoked` event contains only the current delegate, while the pending delegate that was cancelled is omitted. If there is no current delegate, cancelling a real pending nomination emits `DelegateRevoked(address(0))`, which is indistinguishable from a no-op revocation.

Vulnerable scenario

The following steps lead to the described result:

- ① The owner nominates a replacement delegate while the existing delegate remains active during the cooldown.
- ② The owner discovers that the pending address is incorrect and calls `revokeDelegate` before it becomes active.
- ③ The function clears both the pending nomination and the active delegate.

- ④ Monitoring systems observe only the active delegate's revocation and cannot determine which pending nomination was cancelled.
- ⑤ The owner nominates the original or another valid delegate and waits for the cooldown before normal operation can resume.

Result: Cancelling a pending nomination causes an avoidable operational outage and leaves event-driven monitoring with an incomplete delegate history.

Recommendation

- Add an owner-only `revokeNomination` function that clears only `_pendingDelegate` and `_pendingActiveFrom`.
- Add a dedicated event such as `NominationCancelled(address indexed delegate)` and emit it whenever `revokeNomination` or `revokeDelegate` clears a pending nomination.

References

1. SCSVS G4: Business logic
2. SCSVS G6: Communications
3. SCSVS G8: Denial of service

[LDO-55729910-L02] Codeless targets report successful execution

LOW ACKNOWLEDGED

Retest (2026-08-25)

The vulnerability has been acknowledged as the transactions executed by the delegation contract are meant to have the same characteristics as the EOA (delegates).

Lido team's response: *We intentionally did not add this check, to allow the delegate to call any EOA and transfer funds deliberately. Any delegate can already do this without EDF.*

Occurrences

- DelegationContract.sol#L117-L132

Description

The `execute` function rejects the zero address and the delegation contract itself, but it does not verify that `target` contains code. A low-level call to a codeless address succeeds and re-

turns empty data. Consequently, a mistyped, wrong-network, not-yet-deployed, or destroyed target can appear to have completed a safety-critical operation even though no code ran. Any attached `msg.value` is transferred to that address.

Vulnerable scenario

The following steps lead to the described result:

- ① A delegate prepares an oracle report call with 1 ETH attached but supplies a codeless target address because of a configuration error.
- ② The `execute` function calls the address and receives `success == true` with empty return data.
- ③ The function returns normally, so the keeper records the operation as successful.
- ④ No code executes and the target address receives the 1 ETH.

Result: The intended operation is silently skipped and any attached ETH is irrecoverably sent to the codeless target.

Recommendation

Reject codeless targets for contract-call operations:

```
if (target.code.length == 0) revert TargetNotContract();
```

If direct ETH transfers to EOAs are required, expose them through a separate function so the caller explicitly selects value-transfer behavior.

References

1. SCSVS G4: Business logic
2. SCSVS G6: Communications
3. SCSVS I1: Basic

[LDO-55729910-L03] One-step ownership transfer can lock administration

LOW ACKNOWLEDGED

Retest (2026-08-25)

The vulnerability has been acknowledged as the `setOwner` function is meant to be used only once, post-deployment.

Lido team's response: *Two-step ownership transfer doesn't fit Lido's protocol flow. DSM MUST NOT be connected to the protocol if the admin address is incorrect - that's checked at vote-launch time. setOwner is called once, post-deployment and is never expected to be called again afterward.*

Occurrences

- DepositSecurityModule.sol#L170-L177

Description

The `setOwner` function transfers ownership immediately to `newValue`. Although the function rejects the zero address, it does not require the recipient to accept ownership. A typing error, an address copied for the wrong network, or an address that cannot operate the module permanently replaces the current owner.

The owner controls security-sensitive configuration, including the guardian set and quorum, pause-intent validity period, maximum unvetting batch size, and deposit unpausing. Losing access to the owner role prevents those administrative actions because the unintended recipient is the only address allowed to correct the mistake.

Vulnerable scenario

The following steps lead to the described result:

- ① The current owner intends to transfer administration to a new governance executor.
- ② The owner calls `setOwner` with an incorrect but nonzero address.
- ③ The contract immediately stores that address as `owner` and removes the previous owner's authority.
- ④ The unintended recipient cannot call `setOwner` to restore the correct owner.

Result: Administrative control of the Deposit Security Module is permanently assigned to an unusable or unintended address.

Recommendation

Implement a two-step ownership transfer. The current owner should nominate a `pendingOwner`, and only that address should be able to finalize the transfer through `acceptOwnership`. Allow the current owner to replace or cancel a pending nomination before acceptance.

References

1. SCSVS G4: Business logic

2. SCSVS G5: Access control

[LDO-55729910-L04] Pause signatures can be replayed after unpausing

LOW ACKNOWLEDGED

Retest (2026-08-25)

The vulnerability has been acknowledged because the unpause window is larger than the pause-message invalidation window.

Lido team's response: *This is expected behavior. The unpause window is currently much larger than the pause-message invalidation window, so adding a nonce would be redundant.*

Occurrences

- DepositSecurityModule.sol#L371-L388
- DepositSecurityModule.sol#L396-L399

Description

The `pauseDeposits` function validates a signed pause intent using the guardian address and `blockNumber`, but it does not consume the signature or bind it to a pause cycle. Calls made while deposits are already paused return before signature validation. Consequently, signed calls visible on-chain remain reusable until their block-based validity period expires.

After the owner calls `unpauseDeposits`, anyone holding an unexpired guardian signature can submit it again and restore the paused state. This includes a signature from the original pause transaction and additional guardian signatures submitted while the contract was already paused.

The issue is rated Low because replay requires the owner to unpause before the signed intent expires. The current validity window is approximately 22 hours, and unpausing that quickly during an incident is unlikely. The impact is also limited to restoring the protocol's safety-oriented paused state rather than directly exposing funds.

Attack scenario

The attackers might take the following steps in turn:

- ① Observe a relayed guardian transaction containing a valid `pauseDeposits` signature and its `blockNumber`.

- ② Wait for the owner to call `unpauseDeposits` before the pause intent's approximately 22-hour validity window expires.
- ③ Resubmit the previously observed guardian signature and block number.
- ④ The signature remains valid because the contract stores no nonce or consumed-intent state.
- ⑤ The contract sets `isDepositsPaused` to true again.

Result of the attack: Deposits are paused again until the owner intervenes or the replayed intent expires.

Recommendation

Invalidate each signed pause intent after its first successful use. Include a monotonic nonce or pause-cycle identifier in the signed message, store the consumed value, and increment it when a pause succeeds or when the owner unpauses. Validate signatures before the early return if calls made during an existing pause must also be consumed.

References

1. SCSVS G4: Business logic
2. SCSVS G8: Denial of service
3. SCSVS G9: Blockchain data

6. Recommendations

[LDO-55729910-R01] Bound the delegate cooldown

INFO

ACKNOWLEDGED

Retest (2026-08-25)

The recommendation has been acknowledged because the responsibility for validating contracts' parameters belongs to whoever integrates the EDF.

Lido team's response: *We don't want to hardcode min/max bounds, since we don't know in advance which situations or networks this contract will be deployed on. Responsibility for validating parameters — including the cooldown and owner address — belongs to whoever integrates the EDF.*

Description

The `DelegationContract` constructor accepts the complete `uint256` range for `cooldown_`, and the factory forwards the value without validation. An extremely large value can make `block.timestamp + COOLDOWN` overflow on every nomination or schedule activation so far in the future that delegate rotation is effectively unavailable. Very small nonzero values can also defeat the intended observation period by expiring before the next block.

The immutable cooldown is exposed through `getCooldown`, so deployment reviewers can detect an unreasonable value before granting authority to an instance. This reduces the likelihood of harm, but relying only on external review leaves a preventable configuration error in a non-upgradeable contract.

Here are the affected places:

- `DelegationContract.sol#L58-L68`
- `DelegationContract.sol#L71-L81`
- `DelegationFactory.sol#L13-L16`

Recommendation

Define and enforce minimum and maximum cooldown values in the constructor. Permit a zero cooldown only as an explicit documented mode if immediate rotation is required. Mirror the validation in the factory so invalid configuration fails at the deployment entry point, and add boundary tests for zero, the minimum, the maximum, and values above the maximum.

References

1. SCSVS G4: Business logic
2. SCSVS G7: Arithmetic
3. SCSVS G8: Denial of service

[LDO-55729910-R02] Require the delegation contract interface for guardians

INFO ACKNOWLEDGED

Retest (2026-08-25)

The recommendation has been acknowledged. The team assumes the manual review to be sufficient protection.

Lido team's response: *We rely on manual review before adding any delegation contract to the guardians list, as noted in the report itself — every deployed instance is checked by the team before it's granted any power. Extending the on-chain check to the full IDelegationContract interface wouldn't meaningfully change this, since interface support alone doesn't guarantee correct lifecycle/ownership behavior — a contract could implement the interface, but behave incorrectly.*

Description

The `_addGuardian` function accepts any contract that advertises ERC-1271 support. This verifies that a guardian can expose signature validation, but it does not verify that the guardian implements the lifecycle and ownership behavior required from an Execution Delegation Framework contract. An unrelated or incomplete ERC-1271 wallet can therefore be added after the migration even though the upgrade flow separately validates configured delegation contracts.

Here is the affected validation:

- DepositSecurityModule.sol#L317-L325

Recommendation

Check support for the complete `IDelegationContract` interface instead of only `IERC1271`.

References

1. SCSVS G5: Access control

2. SCSVS I1: Basic

[LDO-55729910-R03] Constrain delegate signature semantics

INFO **ACKNOWLEDGED****Retest (2026-08-25)**

The recommendation has been acknowledged as both, EOAs and contracts, are going to be supported and EIP-7702 can be used for post-quantum migration to support new signature scheme.

Lido team's response: *Constraining EIP-7702 usage doesn't make sense, since it doesn't protect against anything meaningful. If an EOA is compromised, the attacker can sign any message that's valid for the delegation contract regardless. But, if ECDSA is ever replaced with a post-quantum signature scheme, that migration can happen without redeploying the delegation contract — just by installing new code via EIP-7702.*

Description

The `isValidSignature` function delegates verification to `SignatureChecker.isValidSignatureNow`. That library chooses ECDSA verification when the delegate has no code and ERC-1271 verification when the delegate has code. The choice is made at validation time rather than when the delegate is nominated.

An EOA delegate can install EIP-7702 code after nomination. This changes the verification scheme used by the delegation contract without changing its delegate state, waiting through the cooldown, or emitting an event. Previously valid ECDSA signatures can stop working, while delegated code that returns the ERC-1271 magic value can make unrelated signatures appear valid.

Here is the affected verification path:

- `DelegationContract.sol#L138-L146`

Recommendation

Define and enforce the supported delegate types. If EOA delegates are required, reject delegates with code and reject EIP-7702 delegated accounts during validation. If contract delegates are required, record the expected code hash at nomination and verify it before every signature check.

References

1. SCSVS G4: Business logic
2. SCSVS G5: Access control
3. SCSVS I1: Basic

[LDO-55729910-R04] Update the ERC-1271 dependency

INFO ACKNOWLEDGED

Retest (2026-08-25)

The recommendation has been acknowledged as the bug-free dependency version is too recent to adopt confidently.

Lido team's response: *Update the ERC-1271 dependency 5.7.0 is too recent to adopt confidently. 5.5 and 5.6 both contain a known bug related to 32-byte padding.*

Description

The project pins OpenZeppelin Contracts version 5.4.0 and uses its `SignatureChecker` implementation for contract delegates. OpenZeppelin version 5.7.0 changed this path to zero-pad ERC-1271 signature calldata to a 32-byte boundary so the dynamic `bytes` value conforms to ABI encoding requirements.

Here are the affected places:

- `package.json`#L18-L20
- `DelegationContract.sol`#L138-L146

Recommendation

- Upgrade OpenZeppelin Contracts to a reviewed release that contains the ERC-1271 padding correction, re-run the complete test suite, and add an integration test using a strict ERC-1271 wallet.
- Configure automated dependency update notifications and document a regular dependency-review process.

References

1. SCSVS G2: Policies and procedures
2. SCSVS I1: Basic

[LDO-55729910-R05] Emit an event for delegated executions

INFO ACKNOWLEDGED

Retest (2026-08-25)

The recommendation has been acknowledged.

Lido team's response: *Emitting on every execute call seems excessive, given the function doesn't change contract state — it's a pure proxy call.*

Description

The `execute` function is the most privileged operation exposed by the delegation contract, but it emits no event. Monitoring systems must use transaction traces to identify delegate activity, while every other state-changing flow has a dedicated log.

Here is the affected function:

- DelegationContract.sol#L117-L133

Recommendation

Emit an event that identifies the delegate, target, transferred value, and a fixed-size digest of the calldata after a successful call. For example:

```
event Executed(address indexed sender, address indexed target, uint256
    value, bytes32 dataHash);

emit Executed(msg.sender, target, msg.value, keccak256(data));
```

Using a digest keeps the event cost bounded while allowing operators to correlate it with known calldata.

References

1. SCSVS G6: Communications

[LDO-55729910-R06] Validate and accept delegate nominations

INFO ACKNOWLEDGED

Retest (2026-08-25)

The recommendation has been acknowledged.

Lido team's response: *We want rotation to be seamless and predictable — the delegate shouldn't be able to stall a rotation simply by not accepting it. We also don't need double confirmation, since the owner retains full control and can change the nominated address at any time. Additionally, requiring the delegate to confirm via an on-chain transaction would mean using the hot key for a purpose outside its intended scope.*

Description

The `nominateDelegate` function verifies that the nominee is nonzero, differs from the owner, and is not already active or pending. It does not require the nominee to accept the role and does not bind a contract nominee to reviewed code. A wrong address can therefore become active after the cooldown, and a counterfactual contract address can remain codeless during review before code is deployed at activation time.

Here are the places where nomination and delegate use do not bind the nominee to an accepted identity or implementation:

- `DelegationContract.sol#L71-L83`
- `DelegationContract.sol#L138-L144`

Recommendation

Use a two-step nomination flow in which the nominated address accepts the role before activation. Define whether EOA delegates, contract delegates, or both are supported. For contract delegates, record the reviewed code hash at nomination and verify that it has not changed before privileged execution or signature validation.

References

1. SCSVS G4: Business logic
2. SCSVS G5: Access control
3. SCSVS I1: Basic

7. Impact on risk classification

Risk classification is based on the one developed by OWASP ¹, however it has been adapted to the immutable and transparent code nature of smart contracts. The Web3 ecosystem forgives much less mistakes than in the case of traditional applications, the servers of which can be covered by many layers of security.

Therefore, the classification is more strict and indicates higher priorities for paying attention to security.

OVERALL RISK SEVERITY				
Impact on risk	HIGH	CRITICAL	HIGH	MEDIUM
	MEDIUM	MEDIUM	MEDIUM	LOW
	LOW	LOW	LOW	INFO
	Likelihood			
		HIGH	MEDIUM	LOW

¹OWASP Risk Rating methodology

8. Long-term best practices

8.1. Use automated tools to scan your code regularly

It's a good idea to incorporate automated tools (e.g. slither) into the code writing process. This will allow basic security issues to be detected and addressed at a very early stage.

8.2. Perform threat modeling

Before implementing or introducing changes to smart contracts, perform threat modeling and think with your team about what can go wrong. Set potential targets of the attacker and possible ways to achieve them, keep it in mind during implementation to prevent bad design decisions.

8.3. Use Smart Contract Security Verification Standard

Use proven standards to maintain a high level of security for your contracts. Treat individual categories as checklists to verify the security of individual components. Expand your unit tests with selected checks from the list to be sure when introducing changes that they did not affect the security of the project.

8.4. Discuss audit reports and learn from them

The best guarantee of security is the constant development of team knowledge. To use the audit as effectively as possible, make sure that everyone in the team understands the mistakes made. Consider whether the detected vulnerabilities may exist in other places, audits always have a limited time and the developers know the code best.

8.5. Monitor your and similar contracts

Use the tools available on the market to monitor key contracts (e.g. the ones where user's tokens are kept). If you have used code from another project, monitor their contracts as well and introduce procedures to capture information about detected vulnerabilities in their code.



Damian Rusinek

Smart Contracts Auditor

@drdr_zz

damian.rusinek@composable-security.com



Paweł Kuryłowicz

Smart Contracts Auditor

@wh01s7

pawel.kurylowicz@composable-security.com

